SHOPBONUS

25 executable E2E journeys 175 checks + 50 applied improvements Exact values, code and proof

QA execution specification | Tester: Hassan | Laravel owner: Frank | Release approver: Martijn | 26 October 2026

Version 2: 50 implemented review recommendations add two executable checks to each journey. All 175 controls are requirements for Hassan to execute, NOT claims that the backend has already passed. HTTP 200 alone is never PASS.

Non-negotiable: Existing routes and business rules must be discovered from code; example paths and IDs in this PDF are illustrative only. Never invent endpoints or mark an unimplemented, unsupported or credential-blocked feature as PASS.

Tests	Focus
01-05	Infrastructure, accounts, merchants, isolation, discovery
06-09	Offer codes, concurrency, Answer24, claims
10-13	Daisycon, TradeTracker, wallet and merchant finance
14-19	Ten connectors, products, banners, attribution
20-25	Vouchers, Mollie, support, CMS, security

Status key: PASS = actual behavior and downstream proof match; FAIL = wrong result; BLOCKED = inaccessible credential/environment; NOT IMPLEMENTED = required feature absent; UNTESTED = no executed evidence; STALE = evidence belongs to an older SHA.

1. Evidence and no-skip protocol

Each check below receives one unique step ID (e.g. SB-21.03), its executable test name and a machine-readable result. A green group means all checks plus relevant negative tests passed on the recorded code SHA.

For EVERY check save: UTC timestamp; run/step ID; SHA; staging environment; actor/tenant; expected and actual assertions; masked request/response and request_id; DB before/after; relevant cent-ledger/code IDs; provider status; webhook/queue/audit records; browser screenshot/trace where visible; evidence file URI and SHA-256.

Coverage gate: Export the Laravel route list, controllers/actions, Policies, Jobs, Events, Listeners, schedule, webhooks and frontend calls; map all to approved tests. APIs not used by the current frontend are still inventoried. For each route include success, input validation, anonymous authentication and role/cross-tenant failures where applicable. Add tests beyond these 25 when needed.

Integrity gate: Fixtures run on isolated staging with real DB, Redis, queue workers, scheduler, Reverb and provider TEST accounts. Unit tests or mocks supplement but cannot substitute a real provider-backed E2E. Production systems, funds and user data are out of scope.

Evidence filename convention: SB-21.03_2026-10-26T091500Z_a1b2c3d_mollie-payment.json; source screenshots use the same prefix. A scenario cannot be PASS if any mandatory evidence is missing.

2. Suggested fixture values (staging only)

Fixture	Exact test input
Shop A / Merchant A	shop-a.test | merchant-a@example.test | id 101
Shop B / Merchant B	shop-b.test | merchant-b@example.test | id 202
Consumer A / B	consumer-a@example.test | consumer-b@example.test
Core offer	Offer id 77 | €25.00 voucher | €100.00 minimum
Money	Store integer cents: €25=2500, €9=900, €16=1600
Critical race	One final code | two concurrent user requests
Provider keys	Separate Mollie TEST / affiliate test / connector sandbox credentials
Clock	Frozen UTC time for day 14 / 30 / 60 / 80 / 90 / 12 months

01 Deployment and health

Setup / exact values: Fresh isolated staging with commit a1b2c3d, DB, Redis, scheduler and Reverb running.

[ ] SB-01.01 Run migrations and verified seed twice; second run must not duplicate reference data.

[ ] SB-01.02 Start HTTP, real queue worker, scheduler and Reverb; execute their health checks.

[ ] SB-01.03 Visit public, merchant, consumer and Super Admin pages with distinct sessions.

[ ] SB-01.04 Confirm DB indexes, constraints, queue failures=0 and no unexplained HTTP 500.

[ ] SB-01.05 Record code SHA, PHP/Laravel version, provider mode and reversible test-fixture cleanup.

[ ] SB-01.06 Compare application and database clocks to UTC, recording drift and timezone configuration.

[ ] SB-01.07 Restart workers after a deployment and prove old workers cannot process incompatible new job schemas.

PASS: Expected: boot succeeds; build SHA matches; no unauthorized admin access.

PROOF: Evidence: migration output, route hash, health requests, worker/schedule/Reverb logs, screenshots.

Result: [ ] PASS [ ] FAIL [ ] BLOCKED [ ] NOT IMPLEMENTED [ ] UNTESTED Evidence link: _____

02 Consumer identity lifecycle

Setup / exact values: consumer-a@example.test; wrong OTP 000000; expired token; passwords changed.

[ ] SB-02.01 Register consumer A; assert one user row and one email-verification delivery.

[ ] SB-02.02 Verify real emailed OTP/token; login; update profile/preferences; logout; login again.

[ ] SB-02.03 Exercise reset password, old password rejected, new password accepted, revoke sessions.

[ ] SB-02.04 Duplicate address, invalid/expired OTP, five rapid invalid attempts: reject and rate-limit.

[ ] SB-02.05 Anonymous/other consumer wallet fetch: 401/403 and zero sensitive body fields.

[ ] SB-02.06 Prevent account enumeration: existing and unknown emails produce indistinguishable public responses.

[ ] SB-02.07 Issue two reset tokens; redeem one, reject reuse and invalidate old sessions per configured policy.

PASS: Expected: only verified rightful consumer accesses account; deterministic errors.

PROOF: Evidence: masked requests, mail test inbox, auth DB change, rate-limit logs, audit.

Result: [ ] PASS [ ] FAIL [ ] BLOCKED [ ] NOT IMPLEMENTED [ ] UNTESTED Evidence link: _____

03 Merchant onboarding and ownership

Setup / exact values: merchant-a@example.test; invitation lifetime 14 days; shop domain shop-a.test.

[ ] SB-03.01 Sign up merchant directly AND separately via signed Answer24 owner invitation.

[ ] SB-03.02 Prefill website/Google metadata; edit name/contact; verify business email/domain ownership.

[ ] SB-03.03 Complete three configured onboarding steps and accept versioned terms.

[ ] SB-03.04 Try invitation after 14 days, second redemption, forged owner and wrong domain.

[ ] SB-03.05 Ensure unverified merchant cannot publish offers or see other merchants.

[ ] SB-03.06 Change signed invitation owner/tenant claims and verify tampering causes a logged denial.

[ ] SB-03.07 Change verified merchant domain/email and require ownership reverification before access changes.

PASS: Expected: only verified owner activates shop; single-use invite is enforced.

PROOF: Evidence: invite signature ID, onboarding states, email/domain challenge and audit.

Result: [ ] PASS [ ] FAIL [ ] BLOCKED [ ] NOT IMPLEMENTED [ ] UNTESTED Evidence link: _____

04 Cross-tenant and RBAC

Setup / exact values: merchant A shop_id=101; merchant B shop_id=202; consumer A/B.

[ ] SB-04.01 Merchant A creates offer; B guesses A shop_id, offer_id, file_id and import ID.

[ ] SB-04.02 Attempt GET/POST/PATCH/DELETE and bulk export using swapped foreign IDs.

[ ] SB-04.03 Anonymous access must get 401; authenticated forbidden actions get 403/404 consistently.

[ ] SB-04.04 Repeat with consumer and Answer24 service token lacking permission.

[ ] SB-04.05 Super Admin authorized override must create attributable audit record.

[ ] SB-04.06 Attempt nested object BOLA: permitted shop path with another merchant offer ID.

[ ] SB-04.07 Reject mass assignment of role, tenant_id, approval_status and other protected properties.

PASS: Expected: no cross-tenant data, state changes, even in error payloads.

PROOF: Evidence: tenant snapshots before/after, 401/403 traces, audit actor IDs.

Result: [ ] PASS [ ] FAIL [ ] BLOCKED [ ] NOT IMPLEMENTED [ ] UNTESTED Evidence link: _____

05 Shops, offers and public discovery

Setup / exact values: shop A with three offers: approved, paused, expired.

[ ] SB-05.01 Create shop/three offers; submit for moderation, reject one and approve one.

[ ] SB-05.02 Publish, pause, suspend and reactivate while preserving histories.

[ ] SB-05.03 Search and filter by shop/category/status; test pagination and stable SEO slug.

[ ] SB-05.04 Public shop cards show only eligible active offers; paused/expired disappear.

[ ] SB-05.05 Merchant B cannot moderate or change shop A status.

[ ] SB-05.06 Edit an already approved offer and verify required moderation/versioning, not automatic republishing.

[ ] SB-05.07 Measure public cache/search propagation against a stated freshness SLA after approval/pause.

PASS: Expected: public data reflects moderation state; no stale cache after changes.

PROOF: Evidence: before/after public screenshots, response payloads and moderation audit.

Result: [ ] PASS [ ] FAIL [ ] BLOCKED [ ] NOT IMPLEMENTED [ ] UNTESTED Evidence link: _____

06 Offer rules and 500-code import

Setup / exact values: €100 min order; fixed €10 max discount; 500 distinct sample codes.

[ ] SB-06.01 Create percent and fixed-value offers with eligibility, currency, limits and expiry.

[ ] SB-06.02 Import 500 unique codes via paste, CSV and XLSX in separate controlled batches.

[ ] SB-06.03 Inject blank line, malformed code, duplicate in same batch, duplicate in DB.

[ ] SB-06.04 Verify accepted+rejected counts equal input rows and no partial orphan codes.

[ ] SB-06.05 Filter Excel-like sheet by issued/available/rejected and inspect history/export.

[ ] SB-06.06 Import UTF-8 BOM, semicolon CSV and XLSX; prove original input row totals reconcile.

[ ] SB-06.07 Test €99.99 vs €100.00 minimum, zero/negative discounts, 100% and capped percent values.

PASS: Expected: exact code counts and original row error reasons; no duplicate active code.

PROOF: Evidence: source fixture checksum, import report, DB COUNT DISTINCT and sheet screenshot.

Result: [ ] PASS [ ] FAIL [ ] BLOCKED [ ] NOT IMPLEMENTED [ ] UNTESTED Evidence link: _____

07 Code concurrency and expiry

Setup / exact values: One remaining unique code; two simultaneous user requests; 90-day expiry.

[ ] SB-07.01 Reserve final code in simultaneous transactions for consumers A and B.

[ ] SB-07.02 Assert exactly one reservation/issued-code record; loser gets explicit sold-out result.

[ ] SB-07.03 Retry same idempotency key ten times; no extra issuance or debit.

[ ] SB-07.04 Test reusable code eligibility and status transitions; expire with controlled test clock.

[ ] SB-07.05 Pause offer; block new code issue but preserve already-issued valid voucher.

[ ] SB-07.06 Run 20 parallel claims and force a DB deadlock retry; exactly one unique code is issued.

[ ] SB-07.07 Assert expiry cutoff at exact UTC boundary and one second later, per published rule.

PASS: Expected: exactly-once allocation under race and retries.

PROOF: Evidence: parallel request IDs, unique index, transaction/lock logs, code counts.

Result: [ ] PASS [ ] FAIL [ ] BLOCKED [ ] NOT IMPLEMENTED [ ] UNTESTED Evidence link: _____

08 Answer24 signed handoff

Setup / exact values: signed contact ref ACT-001; shop 101; offer 77; idempotency key IK-001.

[ ] SB-08.01 Accept authenticated scoped Answer24 request for exact shop+offer only.

[ ] SB-08.02 Return opaque short URL; claim once; replay same key returns same result.

[ ] SB-08.03 Reject unsigned body, forged signature, expired timestamp and unauthorized shop.

[ ] SB-08.04 Ambiguous offer selection must fail; never silently choose another offer.

[ ] SB-08.05 Inspect public URL and logs for leaked customer PII, tokens and internal IDs.

[ ] SB-08.06 Verify signature canonicalization, timestamp skew, key rotation and safe HMAC comparison.

[ ] SB-08.07 Reuse same idempotency key with modified offer; reject conflict without second allocation.

PASS: Expected: one handoff, no duplicate code consumption or identity disclosure.

PROOF: Evidence: signature verifier trace, link record, event logs and count delta.

Result: [ ] PASS [ ] FAIL [ ] BLOCKED [ ] NOT IMPLEMENTED [ ] UNTESTED Evidence link: _____

09 Claim and timed reminders

Setup / exact values: consumer new/existing; day 30/60/80 reminders; day 90 expiry.

[ ] SB-09.01 Invite new consumer; verify OTP; claim exact offer once and create one account.

[ ] SB-09.02 Existing verified account claims without duplicate user; link to same wallet.

[ ] SB-09.03 Reject guessed, used, revoked and expired claim token.

[ ] SB-09.04 Time-travel scheduler to days 30, 60, 80; send once each to still-unclaimed.

[ ] SB-09.05 Claim before next reminder and verify later reminders suppressed; day 90 expires.

[ ] SB-09.06 Normalize mixed-case/whitespace emails and run simultaneous signups; only one identity remains.

[ ] SB-09.07 Test duplicate cron executions, timezone change and notification opt-out precedence.

PASS: Expected: one claim maximum, deduplicated communications and precise expiry.

PROOF: Evidence: mail delivery IDs, schedule runs, token records and DB snapshots.

Result: [ ] PASS [ ] FAIL [ ] BLOCKED [ ] NOT IMPLEMENTED [ ] UNTESTED Evidence link: _____

10 Daisycon tracked earning

Setup / exact values: cashback-approved advertiser; click sub_id CA-001; duplicate transaction T-001.

[ ] SB-10.01 Import only explicitly cashback-approved program; disallowed program never visible.

[ ] SB-10.02 Create consented tracked click, safe redirect, unique first-party sub-ID.

[ ] SB-10.03 Ingest provider-backed test transaction; match click, shopper, shop and currency.

[ ] SB-10.04 Repeat import/event twice; exact same provider transaction creates one conversion.

[ ] SB-10.05 Unmatched/unsupported status goes to manual review, not invented wallet credit.

[ ] SB-10.06 Reconcile provider gross/net commission, currency and rounding without speculative credits.

[ ] SB-10.07 Block open redirects and unauthenticated/no-consent click tracking with audit evidence.

PASS: Expected: correct attribution and exactly-once earning.

PROOF: Evidence: provider test record, raw sanitized import, conversion unique key and ledger.

Result: [ ] PASS [ ] FAIL [ ] BLOCKED [ ] NOT IMPLEMENTED [ ] UNTESTED Evidence link: _____

11 TradeTracker independent earning

Setup / exact values: TradeTracker click TT-002, paginated response and duplicate callbacks.

[ ] SB-11.01 Verify account-specific advertiser permission and tracking reference format.

[ ] SB-11.02 Perform actual authorized tracking URL and provider transaction retrieval.

[ ] SB-11.03 Exercise pagination/cursors, reconnect, changed status and repeated callback.

[ ] SB-11.04 Inject temporary 429/503 and verify bounded retry, checkpoint, alert.

[ ] SB-11.05 Reject Daisycon-shaped fields where TradeTracker mapping differs.

[ ] SB-11.06 Inject repeated/truncated pagination pages and verify no missing or duplicated conversions.

[ ] SB-11.07 Reverse provider status later and prove immutable history plus consistent ledger/reporting.

PASS: Expected: independent mapping, no dropped or duplicate conversions.

PROOF: Evidence: provider IDs, paginated fetch logs, warning and ledger delta.

Result: [ ] PASS [ ] FAIL [ ] BLOCKED [ ] NOT IMPLEMENTED [ ] UNTESTED Evidence link: _____

12 Wallet credit, expiry and rejection

Setup / exact values: conversion +€10.00 (1000 cents); expiry +12 months; oldest-first spending.

[ ] SB-12.01 Credit eligible verified conversion exactly once and show spendable €10.00 immediately.

[ ] SB-12.02 Repeat provider notification ten times; wallet remains 1000 cents, one ledger entry.

[ ] SB-12.03 Credit second older/newer lot and spend; consume oldest expiry first.

[ ] SB-12.04 Simulate provider rejection after wallet spend; log loss/report without negative double debit.

[ ] SB-12.05 Expire remaining credited lot on 12-month boundary using deterministic UTC clock.

[ ] SB-12.06 Test one-cent, zero, maximum and overflow-sized credit inputs using integer arithmetic.

[ ] SB-12.07 Race two simultaneous wallet purchases against one €10 lot; never overspend 1000 cents.

PASS: Expected: integer cents, no floating rounding and no pending wallet bucket.

PROOF: Evidence: ledger invariant SUM(credits)-SUM(debits), provider history and lot order.

Result: [ ] PASS [ ] FAIL [ ] BLOCKED [ ] NOT IMPLEMENTED [ ] UNTESTED Evidence link: _____

13 Merchant liability and invoices

Setup / exact values: Order €100.00; cashback €10.00; fee €3.00; merchant liability €13.00.

[ ] SB-13.01 Record paid direct eligible order; calculate merchant liability exactly 1300 cents.

[ ] SB-13.02 Post one 1000-cent shopper credit and one 300-cent fee per authorized order.

[ ] SB-13.03 Reject duplicate paid callback and fake order total/currency.

[ ] SB-13.04 Simulate merchant insufficient funds; follow actual configured pause/dunning policy.

[ ] SB-13.05 Check refunds, Moneybird invoice and Mollie top-up only where implemented.

[ ] SB-13.06 Reconcile €100 order to 1000-cent cashback, 300-cent fee and 1300-cent liability.

[ ] SB-13.07 Test €40 partial refund and apply only approved versioned adjustment rules.

PASS: Expected: no unexplained merchant debt or contradictory affiliate split.

PROOF: Evidence: versioned terms, order receipt, invoice draft, ledger and funding provider.

Result: [ ] PASS [ ] FAIL [ ] BLOCKED [ ] NOT IMPLEMENTED [ ] UNTESTED Evidence link: _____

14 All ten connector installations

Setup / exact values: Shopify, WooCommerce, Wix, BigCommerce, Ecwid, Lightspeed C, PrestaShop, OpenCart, Shopware 6, Adobe Commerce.

[ ] SB-14.01 For EACH connector create separate authorized test shop; OAuth/API connection is real.

[ ] SB-14.02 Record scopes, shop identity, region/version and secret storage; never leak tokens.

[ ] SB-14.03 Exercise reconnect, revoked scope, expired token and wrong-merchant binding.

[ ] SB-14.04 Exercise uninstall/disconnect and invalidated credentials; never keep a false green.

[ ] SB-14.05 Mark each platform CONNECTED/LIMITED/UNSUPPORTED with separate provider proof.

[ ] SB-14.06 For each of ten connectors, verify installation callback authenticity and least-privilege scopes.

[ ] SB-14.07 For each of ten connectors, revoke token/uninstall and prove stale credentials stop working.

PASS: Expected: ten subruns; absence of live test tenant is BLOCKED, not PASS.

PROOF: Evidence: 10 connection records, provider-side screenshots, scope and revoke logs.

Result: [ ] PASS [ ] FAIL [ ] BLOCKED [ ] NOT IMPLEMENTED [ ] UNTESTED Evidence link: _____

15 All ten connector orders/refunds

Setup / exact values: Paid order €94.00, unpaid order, €12.00 refund; each of 10 platforms.

[ ] SB-15.01 For EACH connected platform fetch real test paid order and payment status.

[ ] SB-15.02 Map external order ID, taxes/net/gross, currency, tenant and line items.

[ ] SB-15.03 Unpaid/canceled orders never earn; refund adjusts correct conversion reporting.

[ ] SB-15.04 Deliver webhook twice/out of order OR execute documented polling fallback.

[ ] SB-15.05 Disconnect/reconnect during sync; checkpoints recover without duplicate ledger.

[ ] SB-15.06 For each connector, reuse an external order ID across two shops; composite keys prevent collision.

[ ] SB-15.07 For each connector, replay partial/multiple refunds without exceeding original paid amount.

PASS: Expected: ten separate provider-backed order+refund subruns.

PROOF: Evidence: 10 receipts, provider payloads, normalized DB state and sync checkpoints.

Result: [ ] PASS [ ] FAIL [ ] BLOCKED [ ] NOT IMPLEMENTED [ ] UNTESTED Evidence link: _____

16 Product and coupon syncing

Setup / exact values: SKU SKU-001; 3 products; one native coupon with 10% discount.

[ ] SB-16.01 Sync product SKU, photos, price, category, stock, URL and storefront merchant identity.

[ ] SB-16.02 Update price/stock and delete product; ensure pagination handles all records.

[ ] SB-16.03 Create same SKU in merchant B; no cross-shop collision or overwritten asset.

[ ] SB-16.04 Read native coupon rules and validity, separating checkout coupon from wallet voucher.

[ ] SB-16.05 Show accurate product/offer on consumer public page after refresh.

[ ] SB-16.06 Reconcile tax inclusive/exclusive, price minor units and multi-currency catalog fields.

[ ] SB-16.07 Reject unsafe product-photo URLs (SSRF), invalid MIME and oversized assets.

PASS: Expected: accurate current catalog; no orphan public deleted product.

PROOF: Evidence: before/after provider SKU, public screenshot and sync deltas.

Result: [ ] PASS [ ] FAIL [ ] BLOCKED [ ] NOT IMPLEMENTED [ ] UNTESTED Evidence link: _____

17 Embed and banner publication

Setup / exact values: real sandbox storefront domain, draft v1 and published v2.

[ ] SB-17.01 Install supported snippet or Shopify App Embed on actual test storefront.

[ ] SB-17.02 Verify verified domain, CSP, escaping and storefront load without script errors.

[ ] SB-17.03 Merchant edits banner privately on storefront, saves draft and previews.

[ ] SB-17.04 Schedule publish, pause and roll back; public sees correct version each time.

[ ] SB-17.05 Unauthorized editor, revoked domain and removed script must not keep publishing.

[ ] SB-17.06 Publish new banner while old version serves; verify atomic activation and isolated previews.

[ ] SB-17.07 Test malicious HTML, script and CSS injection; enforce sanitization/CSP on storefront.

PASS: Expected: no unapproved/public draft and install-uninstall is reversible.

PROOF: Evidence: actual storefront screenshots, DOM trace, publication version and audit.

Result: [ ] PASS [ ] FAIL [ ] BLOCKED [ ] NOT IMPLEMENTED [ ] UNTESTED Evidence link: _____

18 Targeting and analytics accuracy

Setup / exact values: Cart €94, threshold €100, show €6 remaining; 70 views, 7 clicks.

[ ] SB-18.01 Target by URL, category, device, time and eligible consent state.

[ ] SB-18.02 Apply frequency cap and randomized A/B holdout; exclude sensitive checkout views.

[ ] SB-18.03 Show remaining €6 only when live cart integration accurately supports it.

[ ] SB-18.04 Generate exactly 70 qualified visible impressions and 7 CTA clicks: CTR=10%.

[ ] SB-18.05 Check close-X is not a CTA click and bot/hidden views are excluded.

[ ] SB-18.06 Deduplicate client events across reconnect/refresh by immutable event ID.

[ ] SB-18.07 Verify CTR arithmetic: 7/70=10.00%; 6/70=8.57% after explicit rounding.

PASS: Expected: only eligible impressions count; unsupported cart feature not fabricated.

PROOF: Evidence: event IDs, timestamps, analytics query and browser video/screenshot.

Result: [ ] PASS [ ] FAIL [ ] BLOCKED [ ] NOT IMPLEMENTED [ ] UNTESTED Evidence link: _____

19 Click/order/repeat attribution

Setup / exact values: first visit, consent, paid order #1, voucher, paid repeat order #2.

[ ] SB-19.01 Consent -> visible banner -> CTA -> first-party tracked click -> paid order.

[ ] SB-19.02 Create voucher once; use it at repeat purchase, including cross-shop scenario.

[ ] SB-19.03 Ensure first/second conversions and sources preserve immutable linked IDs.

[ ] SB-19.04 Refund repeat order; adjust merchant/network reporting without invented sale.

[ ] SB-19.05 Without consent/trace, label Unattributed rather than backfill invented source.

[ ] SB-19.06 Apply documented attribution window/source precedence and consent withdrawal rules.

[ ] SB-19.07 Compare attributed revenue only against actual verified paid orders; exclude pending.

PASS: Expected: no double attribution, traceable shop-level ROI.

PROOF: Evidence: click IDs, provider order receipts, voucher and reporting query.

Result: [ ] PASS [ ] FAIL [ ] BLOCKED [ ] NOT IMPLEMENTED [ ] UNTESTED Evidence link: _____

20 Wallet-only voucher checkout

Setup / exact values: wallet €30.00, voucher €25.00, remainder €5.00.

[ ] SB-20.01 Start with one 3000-cent wallet lot; select published eligible €25 voucher.

[ ] SB-20.02 Check entitlement/stock then reserve and debit exactly 2500 cents atomically.

[ ] SB-20.03 Issue one code in My Vouchers, allow copy and verify real supported checkout.

[ ] SB-20.04 Differentiate ISSUED from MERCHANT-VERIFIED REDEEMED; enforce expiry.

[ ] SB-20.05 Check permitted gift transfer once; reject cash-out or IBAN endpoints.

[ ] SB-20.06 Start wallet debit with zero voucher stock; verify atomic rollback leaves balance unchanged.

[ ] SB-20.07 Separate ISSUED from merchant-verified REDEEMED and reject unsupported checkout claims.

PASS: Expected: 500-cent wallet balance, one code, one debit, no fiat withdrawal.

PROOF: Evidence: wallet ledger IDs, code table, consumer screenshot and external validation.

Result: [ ] PASS [ ] FAIL [ ] BLOCKED [ ] NOT IMPLEMENTED [ ] UNTESTED Evidence link: _____

21 Mollie TEST split payment

Setup / exact values: voucher €25.00; wallet €9.00; Mollie payment EUR 16.00.

[ ] SB-21.01 Reserve exactly 900 wallet cents; create provider TEST payment for 1600 cents.

[ ] SB-21.02 Complete hosted TEST checkout; server retrieves Mollie payment status from API.

[ ] SB-21.03 On verified paid: capture wallet 900 once, payment 1600 once, issue code once.

[ ] SB-21.04 Check consumer wallet/voucher screen, provider ID and merchant accounting.

[ ] SB-21.05 Repeat redirect, webhook and receipt fetch; no duplicate effects.

[ ] SB-21.06 Reconcile 900-cent wallet reservation and independent Mollie TEST EUR 16.00 status.

[ ] SB-21.07 Crash worker after Mollie PAID before code issue; retry yields one code and one debit.

PASS: Expected: exactly €25 funded, code only after verified provider paid.

PROOF: Evidence: sanitized Mollie TEST payment, DB ledger and webhook event IDs.

Result: [ ] PASS [ ] FAIL [ ] BLOCKED [ ] NOT IMPLEMENTED [ ] UNTESTED Evidence link: _____

22 Mollie failures and recovery

Setup / exact values: €16 attempt failures: canceled, expired, failed, duplicate paid event.

[ ] SB-22.01 For each failed/canceled/expired state, release €9 reservation once, no code issue.

[ ] SB-22.02 Return from redirect BEFORE webhook: UI pending until server verifies state.

[ ] SB-22.03 Duplicate provider callback ten times; only one idempotent transition.

[ ] SB-22.04 Wrong amount, currency, payment ID, tampered owner: reject and audit.

[ ] SB-22.05 Simulate race, refund and chargeback; require approved reconciliation policy.

[ ] SB-22.06 Prove allowed payment state transitions and forbid paid-to-pending rollback.

[ ] SB-22.07 Forge callback ID/body; always fetch authoritative Mollie status before credit/issue.

PASS: Expected: no early code, negative wallet or double charge/issue.

PROOF: Evidence: each Mollie status, state machine transitions, failed reconciliation alerts.

Result: [ ] PASS [ ] FAIL [ ] BLOCKED [ ] NOT IMPLEMENTED [ ] UNTESTED Evidence link: _____

23 Unusable code and support review

Setup / exact values: issued voucher code X; screenshot image/png 250KB.

[ ] SB-23.01 Customer opens case linked to exact owned voucher/offer and uploads screenshot.

[ ] SB-23.02 Validate file MIME, size, malware path and inaccessible private storage.

[ ] SB-23.03 AI may draft advice only; no permission to mint voucher or wallet movement.

[ ] SB-23.04 Authorized human approves replacement once from same eligible shop inventory.

[ ] SB-23.05 Repeat request, forged merchant, unrelated voucher and altered upload all denied.

[ ] SB-23.06 Validate uploaded file sniffing, malware status, private URL expiry and authorized access.

[ ] SB-23.07 Exhaust replacement stock, repeat human approval and ensure no double credit/replacement.

PASS: Expected: one replacement maximum after human approval; original history intact.

PROOF: Evidence: private upload metadata, ticket, approval actor and old/new code linkage.

Result: [ ] PASS [ ] FAIL [ ] BLOCKED [ ] NOT IMPLEMENTED [ ] UNTESTED Evidence link: _____

24 CMS, languages and notifications

Setup / exact values: Home, Webshops, About, FAQ; NL/EN; draft version 1 -> publish 2.

[ ] SB-24.01 Super Admin inline edits NL/EN copy, asset/SEO metadata; preview without public change.

[ ] SB-24.02 Publish, compare public language render, then roll back exact prior version.

[ ] SB-24.03 Merchant/consumer attempts same CMS operations: 403 without DB change.

[ ] SB-24.04 Trigger signup, claim, expiry and Mollie messages; inspect correct template/recipient.

[ ] SB-24.05 Check unsubscribe, email dedupe, unsafe HTML/image URL blocked, notification audit.

[ ] SB-24.06 Inject localized rich-text XSS, unsafe image and bad canonical; verify safe NL/EN fallback.

[ ] SB-24.07 Publish from two editor tabs simultaneously; optimistic version lock prevents lost update.

PASS: Expected: versioned safe copy; no duplicate notices or cross-language mix.

PROOF: Evidence: page screenshots, diff, mail provider IDs and audit log.

Result: [ ] PASS [ ] FAIL [ ] BLOCKED [ ] NOT IMPLEMENTED [ ] UNTESTED Evidence link: _____

25 Security, resilience and restore

Setup / exact values: attack cases 401/403/419/422/429/500; Redis crash; restored staging backup.

[ ] SB-25.01 Exercise CSRF, XSS, SQLi-style input, mass assignment, SSRF and stolen/revoked session.

[ ] SB-25.02 Verify protected routes do not disclose PII/secrets and log sanitized failures.

[ ] SB-25.03 Stop worker during job, restart, retry safely without duplicate ledger/code.

[ ] SB-25.04 Disconnect provider and Reverb, restart scheduler; bounded alerts, graceful degraded UI.

[ ] SB-25.05 Restore isolated staging DB backup; compare balances, unique codes, audit and p95.

[ ] SB-25.06 Prove backup RPO/RTO and post-restore queue, wallet, code and audit checksums.

[ ] SB-25.07 Run object/function authorization attacks and 429/500 paths; ensure redacted logs.

PASS: Expected: recovery reconciles totals; no silent loss, no critical/high open defect.

PROOF: Evidence: attack traces, job attempts, alert IDs, backup/restore and invariant checks.

Result: [ ] PASS [ ] FAIL [ ] BLOCKED [ ] NOT IMPLEMENTED [ ] UNTESTED Evidence link: _____

3. Fifty review tips - all incorporated as executable checks

These fifty specific improvements have been applied above as checks SB-01.06 through SB-25.07. Each must produce its own machine-verifiable result and proof, and may not inherit a green status from another check.

01. Test 01 - Compare application and database clocks to UTC, recording drift and timezone configuration.
02. Test 01 - Restart workers after a deployment and prove old workers cannot process incompatible new job schemas.
03. Test 02 - Prevent account enumeration: existing and unknown emails produce indistinguishable public responses.
04. Test 02 - Issue two reset tokens; redeem one, reject reuse and invalidate old sessions per configured policy.
05. Test 03 - Change signed invitation owner/tenant claims and verify tampering causes a logged denial.
06. Test 03 - Change verified merchant domain/email and require ownership reverification before access changes.
07. Test 04 - Attempt nested object BOLA: permitted shop path with another merchant offer ID.
08. Test 04 - Reject mass assignment of role, tenant_id, approval_status and other protected properties.
09. Test 05 - Edit an already approved offer and verify required moderation/versioning, not automatic republishing.
10. Test 05 - Measure public cache/search propagation against a stated freshness SLA after approval/pause.
11. Test 06 - Import UTF-8 BOM, semicolon CSV and XLSX; prove original input row totals reconcile.
12. Test 06 - Test €99.99 vs €100.00 minimum, zero/negative discounts, 100% and capped percent values.
13. Test 07 - Run 20 parallel claims and force a DB deadlock retry; exactly one unique code is issued.
14. Test 07 - Assert expiry cutoff at exact UTC boundary and one second later, per published rule.
15. Test 08 - Verify signature canonicalization, timestamp skew, key rotation and safe HMAC comparison.
16. Test 08 - Reuse same idempotency key with modified offer; reject conflict without second allocation.
17. Test 09 - Normalize mixed-case/whitespace emails and run simultaneous signups; only one identity remains.
18. Test 09 - Test duplicate cron executions, timezone change and notification opt-out precedence.
19. Test 10 - Reconcile provider gross/net commission, currency and rounding without speculative credits.
20. Test 10 - Block open redirects and unauthenticated/no-consent click tracking with audit evidence.
21. Test 11 - Inject repeated/truncated pagination pages and verify no missing or duplicated conversions.
22. Test 11 - Reverse provider status later and prove immutable history plus consistent ledger/reporting.
23. Test 12 - Test one-cent, zero, maximum and overflow-sized credit inputs using integer arithmetic.
24. Test 12 - Race two simultaneous wallet purchases against one €10 lot; never overspend 1000 cents.
25. Test 13 - Reconcile €100 order to 1000-cent cashback, 300-cent fee and 1300-cent liability.
26. Test 13 - Test €40 partial refund and apply only approved versioned adjustment rules.
27. Test 14 - For each of ten connectors, verify installation callback authenticity and least-privilege scopes.
28. Test 14 - For each of ten connectors, revoke token/uninstall and prove stale credentials stop working.
29. Test 15 - For each connector, reuse an external order ID across two shops; composite keys prevent collision.
30. Test 15 - For each connector, replay partial/multiple refunds without exceeding original paid amount.
31. Test 16 - Reconcile tax inclusive/exclusive, price minor units and multi-currency catalog fields.
32. Test 16 - Reject unsafe product-photo URLs (SSRF), invalid MIME and oversized assets.
33. Test 17 - Publish new banner while old version serves; verify atomic activation and isolated previews.
34. Test 17 - Test malicious HTML, script and CSS injection; enforce sanitization/CSP on storefront.
35. Test 18 - Deduplicate client events across reconnect/refresh by immutable event ID.
36. Test 18 - Verify CTR arithmetic: 7/70=10.00%; 6/70=8.57% after explicit rounding.
37. Test 19 - Apply documented attribution window/source precedence and consent withdrawal rules.
38. Test 19 - Compare attributed revenue only against actual verified paid orders; exclude pending.
39. Test 20 - Start wallet debit with zero voucher stock; verify atomic rollback leaves balance unchanged.
40. Test 20 - Separate ISSUED from merchant-verified REDEEMED and reject unsupported checkout claims.
41. Test 21 - Reconcile 900-cent wallet reservation and independent Mollie TEST EUR 16.00 status.
42. Test 21 - Crash worker after Mollie PAID before code issue; retry yields one code and one debit.
43. Test 22 - Prove allowed payment state transitions and forbid paid-to-pending rollback.
44. Test 22 - Forge callback ID/body; always fetch authoritative Mollie status before credit/issue.
45. Test 23 - Validate uploaded file sniffing, malware status, private URL expiry and authorized access.
46. Test 23 - Exhaust replacement stock, repeat human approval and ensure no double credit/replacement.

47. Test 24 - Inject localized rich-text XSS, unsafe image and bad canonical; verify safe NL/EN fallback.
48. Test 24 - Publish from two editor tabs simultaneously; optimistic version lock prevents lost update.
49. Test 25 - Prove backup RPO/RTO and post-restore queue, wallet, code and audit checksums.
50. Test 25 - Run object/function authorization attacks and 429/500 paths; ensure redacted logs.

4. API Inventory + protected documentation portal

Recommendation: Host a self-managed, read-only OpenAPI portal at https://be.shopbonus.nl/qa (example DNS only) or qa.shopbonus.nl. Serve Swagger UI or Stoplight Elements with a locally hosted validated OpenAPI document; add ShopBonus QA coverage and evidence via a separate authenticated dashboard component. Swagger/Stoplight visualize documentation; neither proves test execution by itself.

Protection: Reverse proxy + Laravel session/Sanctum authentication + server-side Super Admin permission checks for the HTML shell AND the OpenAPI JSON, coverage API, run evidence, exports and asset endpoints. Protect WebSockets too. An unsigned request, merchant or consumer gets 401/403 with no spec bytes. Enable MFA/short session, CSRF for mutations, rate limit, secure cookies, no-index/no-store, strict CSP, restricted CORS, redacted logs. Prefer VPN/IP allowlist as an additional layer, not a replacement for login.

No public Try it out: Disable interactive calls by default, especially financial/DELETE/webhooks. When enabled for approved Super Admin, target allowlisted staging only, with short-lived scoped credentials, clear destructive-action guard, CSRF and audit of each execution. Never expose production credentials or token examples in the spec.

Discover: `php artisan route:list --json` -> normalize method + URI + action + middleware; compare against OpenAPI operations, frontend call sites and jobs/events. The OpenAPI file is a reviewed contract, not generated blindly from route names. Its version must match the deployed SHA. Compare both directions: route exists without spec; spec exists without route; route/spec without executable check; stale check; orphan/unused route. Unused is a review flag, not automatic deletion.

Example inventory row: `POST /api/v1/vouchers/checkout | role consumer | Policy canPurchase | wallet+Mollie state | SB-20/SB-21/SB-22 | 5 assertions | last SHA | latest PASS | evidence URLs`. Here the path is an illustration and MUST be replaced with real code-discovered URI.

Nightly comparison: Deploy SHA changes invalidate PASS for impacted code; newly discovered route = UNTESTED until mapped, then run targeted tests and full nightly E2E. Never silently ignore 404s or count skipped tests.

5. Concrete executable samples (adapt to actual routes)

SAMPLE A - Postman after-response assertion (replace route/schema with inspected implementation):

```
pm.test("expected status", () => pm.response.to.have.status(201));
const body = pm.response.json();
pm.test("real voucher ID", () => {
  pm.expect(body.data.id).to.be.a("string").and.not.empty;
});
pm.test("provider amount is integer cents", () => {
  pm.expect(body.data.amount_cents).to.eql(1600);
});
pm.collectionVariables.set("voucher_id", body.data.id);
```

SAMPLE B - Unauthorized tenant access (expected 403 or intentionally concealed 404, never 2xx):

```
const status = pm.response.code;
pm.test("tenant B denied", () => pm.expect([403,404]).to.include(status));
pm.test("no foreign shop data", () => {
  pm.expect(pm.response.text()).not.to.include("merchant-a@example.test");
});
// Verify unchanged DB row and audit denial via staging-only inspector.
```

SAMPLE C - Laravel Pest logic for duplicate callback invariants (illustrative code):

```
it('does not double-issue a paid voucher', function () {
    $payment = TestPayment::factory()->paid()->create();
    app(VerifiedPaymentHandler::class)->handle($payment->id);
    app(VerifiedPaymentHandler::class)->handle($payment->id);
    expect(Voucher::where('payment_id',$payment->id)->count())->toBe(1);
```

```
    expect(WalletEntry::where('payment_id',$payment->id)
        ->where('kind','debit')->count())->toBe(1);
});
// Replace classes/fields with audited real project implementation.
```

SAMPLE D - Command, real HTTP collection (Postman CLI supports cli/json/junit/html):

```
postman collection run shopbonus-e2e.postman_collection.json \
  -e shopbonus-staging.postman_environment.json \
  -r cli,json,junit,html
# Store sanitized reports in immutable run artifact storage.
```

Important: Never store raw secrets/OTP/passwords, customer PII, full payment payloads or bearer tokens in QA evidence. Use test fixtures and mask sensitive fields before upload. Do not use example passwords in production.

SAMPLE E - Exact cent-ledger invariant, read against staging DB (adjust actual schema):

```
-- Illustrative SQL: do not assume these tables exist.
SELECT owner_id,
 SUM(CASE WHEN direction='credit' THEN amount_cents
          WHEN direction='debit' THEN -amount_cents ELSE 0 END)
   AS computed_balance_cents
FROM wallet_entries WHERE owner_id=:consumer_a
GROUP BY owner_id;
-- Compare result against independently queried wallet balance.
-- Failing inequality is FAIL even if checkout returned HTTP 200.
```

SAMPLE F - Split payment expected state-transition assertions:

```
payment_eur_cents = 2500 - 900 = 1600
PRE_PAYMENT:  wallet_total=3000; reserved=900; spendable=2100
PENDING:      same reservation; no issued code
VERIFIED_PAID: reservation captured once; issued_code_count=1
FAIL/CANCEL:  reservation released once; issued_code_count=0
REPLAY_PAID:  all ledger/code counts unchanged from PAID
NEVER trust browser redirect or unsigned callback for PAID.
```

SAMPLE G - A deterministic 20-client barrier test must verify both application and database state:

```
# Pseudocode: implement via real concurrency harness
create(ONE_AVAILABLE_UNIQUE_CODE)
parallel_barrier(20, request_claim_with_DISTINCT_users)
assert count_successful_claims == 1
assert issued_code_rows_for_offer == 1
assert remaining_available_codes == 0
assert no_wallet_double_debit
store_each_request_id_and_db_transaction_id
```

SAMPLE H - Evidence object contract saved for every SB-XX.YY check:

```
{
 "step_id":"SB-21.03", "git_sha":"<deployed-sha>",
 "environment":"isolated-staging", "actor":"consumer-a",
 "expected":{"wallet_debit_cents":900,"provider_paid_cents":1600},
 "actual":{"wallet_debit_cents":900,"provider_paid_cents":1600},
 "request_id":"<correlation-id>", "proof_refs":["<ledger>","<mollie>"],
 "assertions":7, "failed_assertions":0, "status":"PASS",
 "utc":"2026-10-26T09:15:00Z", "evidence_sha256":"<hash>"
}
```

6. Build the always-current coverage database

Minimal relational model: `qa_api_operations` (method/path/action/sha/auth/tenant), `qa_components` (jobs, listeners, schedules, frontend calls and integration adapters), `qa_test_cases` (immutable versioned executable spec), `qa_test_coverage` (requirement/operation -> test assertion), `qa_runs` (sha, env, start/end, initiator), `qa_results` (step status, expected/actual, request_id), `qa_evidence` (encrypted reference, hash, retention), `qa_findings` (owner, severity, fix SHA/retest). Use constraints and foreign keys to prevent green orphan records.

Dashboard tiles: API routes found / documented / executed / PASS / FAIL / UNTESTED / STALE; jobs/events covered; ten-platform test coverage; financial checks; security exceptions; latest deploy SHA; nightly E2E trend; defects by Frank and Hassan retests. Search route to see source location, related scenarios, proof and current last-run status.

Proof of unused endpoints: Combine static route inventory, frontend call-site references, observed privacy-safe request telemetry and integration/webhook ownership. Label `UNREFERENCED - REVIEW` instead of deleting. A background callback may be intentionally unused by browser clients. Require owner decision and tests before retirement.

Automation cadence: on deploy run targeted contract/Pest and safe staging smoke; nightly run full 25+ scenarios on resettable staging; weekly concurrency/security/backup restore and provider-backed runs; production only read-only health/probe routes with strict budgets. GPT-6 may summarize findings or propose tests but may never fabricate PASS, set release approval or autonomously alter balances/payments.

7. Signed release checklist

[ ] Every discovered route, Policy, job, webhook, scheduler, adapter and frontend action has a test mapping or an explicit reviewed exception.

[ ] All 25 scenario groups completed, plus ten real subruns each for connector install and order/refund, with provider evidence.

[ ] All security negative paths and tenant isolation verified for sensitive routes.

[ ] All money/voucher/codes invariants independently verified from database and provider.

[ ] Every result linked to immutable masked evidence with file hash, date and same deploy SHA.

[ ] No P0/P1 defect, no BLOCKED required provider, no NOT IMPLEMENTED required function, no missing proof.

[ ] Frank fix commits followed by independent Hassan retests and adjacent regression runs.

[ ] Martijn makes final GO/NO-GO decision using signed manifest and risk report.

Signature | Hassan: _____ Frank: _____ Martijn: _____

Official implementation references

Postman CLI reporters: https://learning.postman.com/docs/postman-cli/postman-cli-reporters/

Postman collection runner: https://learning.postman.com/docs/postman-cli/postman-cli-run-collection/

Swagger UI configuration: https://swagger.io/docs/open-source-tools/swagger-ui/usage/configuration/

Stoplight Elements API component: https://docs.stoplight.io/docs/elements/e4dc3dec0afcc-great-api-documentation

Stoplight Spectral OAS rules: https://docs.stoplight.io/docs/spectral/4dec24461f3af-open-api-rules

OWASP broken object authorization: https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/